03 — Mimari ve Teknoloji

Bu doküman MatPortal'ın **teknik mimarisini**, Paspas ERP ile **entegrasyon noktalarını**, **deployment topolojisini** ve teknoloji kararlarının **gerekçelerini** açıklar. Hedef: hem teknik müdür/yatırımcı hem de senior geliştirici okuyabilsin.

1. Mimari Genel Bakış

1.1 Üst Düzey Diyagram

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          KULLANICI KATMANI                               │
├──────────────────────────────────────────────────────────────────────────┤
│  Bayi Web   │  Yönetici   │  Saha Mobil  │  Operatör  │  Sohbet UI  │
│  (Next.js)  │  (Next.js)  │  (Flutter)   │  (Mobile)  │  (Next.js)  │
└──────┬──────────────┬──────────────┬──────────────┬──────────────┬──────┘
       │              │              │              │              │
       └──────────────┴──────────────┴──────────────┴──────────────┘
                                      │ HTTPS (REST + WebSocket)
                                      ↓
┌──────────────────────────────────────────────────────────────────────────┐
│                        API GATEWAY KATMANI                               │
├──────────────────────────────────────────────────────────────────────────┤
│              Nginx (TLS + rate limit + reverse proxy)                    │
└──────────────────────────────────┬───────────────────────────────────────┘
                                   ↓
┌──────────────────────────────────────────────────────────────────────────┐
│                     UYGULAMA KATMANI (Bun runtime)                       │
├──────────────────────────────────────────────────────────────────────────┤
│  Paspas Backend (Fastify 5)                                              │
│  • /admin/*       — mevcut ERP endpoints                                 │
│  • /portal/*      — yeni Faz 6 endpoints                                 │
│  • /tahmin/*      — Faz 2 sipariş tahmin                                 │
│  • /churn/*       — Faz 5 churn radar                                    │
│  • /lead/*        — Faz 3 müşteri keşif                                  │
│  • /sohbet/*      — Faz 8 konversasyonel                                 │
│  • /mlops/*       — Faz 7 model yönetim                                  │
│  • /webhook/*     — dış sistem callback                                  │
└──────┬──────────────┬──────────────┬──────────────┬──────────────────────┘
       │              │              │              │
       ↓              ↓              ↓              ↓
┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────────┐
│ ML       │  │ Crawler  │  │ LLM      │  │ Email / SMS  │
│ Service  │  │ Worker   │  │ Gateway  │  │ / WhatsApp   │
│          │  │          │  │          │  │              │
│ Python   │  │ Crawlee + │  │ Claude / │  │ Brevo /      │
│ FastAPI  │  │ Playwright │  │ OpenAI / │  │ Mailgun /    │
│ scikit / │  │ + Apify  │  │ Groq     │  │ Twilio       │
│ Prophet/ │  │          │  │ multi-   │  │              │
│ XGBoost  │  │          │  │ provider │  │              │
└────┬─────┘  └────┬─────┘  └────┬─────┘  └──────────────┘
     │             │             │
     └─────────────┴─────────────┘
                   ↓
┌──────────────────────────────────────────────────────────────────────────┐
│                          VERİ KATMANI                                    │
├──────────────────────────────────────────────────────────────────────────┤
│  MySQL 8     │ Redis        │ S3 / MinIO       │ MLflow            │
│  (ana DB)    │ (cache+queue)│ (HTML snapshot,  │ (model registry)  │
```

```
│              │              │  model artifact) │                  │
└──────────────────────────────────────────────────────────────────────┘
```

1.2 Bileşen sorumluluğu

Bileşen	Sorumluluk	Teknoloji
Paspas Backend	İş mantığı, REST API, auth, DB CRUD	Fastify 5 + Bun + Drizzle
Bayi Web (Portal)	Bayi UI	Next.js 16 + React 19
Yönetici Web	Mevcut admin panel + yeni modül UI	Next.js 16 + React 19
Saha Mobil	Saha satış ekibi mobil	Flutter
Operatör Mobil	Üretim ekranı (mevcut)	Flutter
ML Service	Model eğitim + tahmin	Python FastAPI + scikit + Prophet + XGBoost
Crawler Worker	Web kazıma + sosyal medya	Crawlee + Playwright + Apify
LLM Gateway	Multi-provider abstraction	Bun + provider SDK'ları
Email/SMS Gateway	Bildirim + outreach	Brevo + Mailgun (yedek)
Cache + Queue	Asenkron iş + cache	Redis + BullMQ
Model Registry	Model versiyonlama	MLflow
Object Storage	Dosya/snapshot/artifact	S3 (cloud) veya MinIO (self-host)

2. Teknoloji Kararları (ve Gerekçeleri)

2.1 Backend — Fastify + Bun

Karar: Paspas backend Fastify 5 + Bun runtime. MatPortal modülleri **aynı projeye yeni route'lar** olarak eklenir.

Gerekçe: - Paspas zaten Fastify+Bun kullanıyor, **yeni stack öğrenmeye gerek yok** - Bun runtime: Node'dan **2-3x hızlı**, native TypeScript, native test - Tek backend → tek auth, tek DB

connection pool, tek deployment - Yeni bir Express/NestJS kurmak fragmentasyon yaratır

Alternatif değerlendirilen: - ❌ Express: Yavaş, Promats projesi modern stack'te - ❌ NestJS: Aşırı katmanlı, küçük takım için overhead - ❌ Microservices (her modül ayrı backend): erken optimizasyon, operasyonel yük büyük - ✅ Fastify monolith: doğru tercih

2.2 Frontend — Next.js 16 + React 19

Karar: İki ayrı Next.js uygulaması: `admin_panel/` (mevcut) ve `dealer_portal/` (yeni).

Gerekçe: - Yönetici ve bayi UI'ları **ayrı izolasyon** ister (farklı kullanıcı, farklı yetki, farklı UX) - Tek Next.js'te iki rol göstermek şişme + güvenlik riski - Aynı backend → kod paylaşımı `_shared/` üzerinden - Tailwind 4 + Shadcn UI Promats projelerinde standart

Frontend stack: - **Next.js 16** App Router - **React 19** server components, suspense - **Redux Toolkit + RTK Query** state + cache - **Shadcn/UI** + **Tailwind 4** stil - **Zod** validation (backend schemas paylaşılır) - **next-intl** çoklu dil hazırlığı (Faz 9 sonrası)

2.3 Veritabanı — MySQL + Drizzle

Karar: Mevcut MySQL 8 devam. Yeni 30+ tablo aynı `promats_erp` DB'ye eklenir.

Gerekçe: - Paspas zaten MySQL + Drizzle ORM - Tek DB → JOIN'ler basit, transaction güçlü - char(36) UUID konvansiyonu Promats projelerinde standart - Migration: seed-based (CLAUDE.md kuralı: ALTER yasak, `0XX_*_schema.sql` güncellenir)

MySQL vs PostgreSQL? - PostgreSQL'in avantajları: JSON sorguları, full-text search, advanced index - Ama: Promats stack'inde MySQL standart, geri dönüş maliyeti yüksek - Eklenebilen JSON desteği MySQL 8 ile yeterli (modern) - ✅ MySQL devam

2.4 ML Service — Python FastAPI (ayrı container)

Karar: Python FastAPI servisi ayrı Docker container'da. Bun backend HTTP üzerinden iletişim.

Gerekçe: - ML kütüphaneleri (scikit-learn, Prophet, XGBoost, SHAP) **Python ekosisteminde** - Bun'da bunları yeniden yazmak imkansız (saatlik / aylık eğitim) - Köprü: REST API - Bun → POST `/train` → Python eğitir, MLflow'a yazar, ID döner - Bun → POST `/predict` → Python tahmin döner (cache'li, <100ms) - Container izolasyonu: Python crash'i Bun'u etkilemez

Alternatif: - ❌ Bun'dan child_process spawn: ölçeklenmez, tek instance - ❌ JS-native ML (TensorFlow.js): Prophet/XGBoost yok, ekosistem zayıf - ✅ FastAPI ayrı container: doğru tercih

Stack detayı:

```
ml_service/
├── Dockerfile
├── requirements.txt    # fastapi, scikit-learn, prophet, xgboost, mlflow, shap
├── app/
│   ├── main.py        # FastAPI app
│   ├── routers/       # /train, /predict, /evaluate
│   ├── models/        # algoritma wrapper'lar
│   ├── pipeline/      # feature engineering
│   └── storage/       # MLflow + S3 client
```

2.5 Crawling — Crawlee + Playwright

Karar: TypeScript native Crawlee + Playwright (ana backend ile aynı dilde).

Gerekçe: - Crawlee Apify ekiplinin: queue, retry, throttle, robots.txt **built-in** - Playwright: JS-rendered modern siteler için (bayilerin çoğu) - TypeScript → mevcut backend ile kod paylaşımı - Apify (sosyal medya): managed scraping, ToS uyumlu

Alternatif: - ❌ Python Scrapy: ayrı dil, ayrı stack - ❌ Puppeteer: Playwright daha modern + cross-browser - ❌ Cheerio only: JS-rendered siteler kazıyamıyor

2.6 LLM — Multi-provider abstraction

Karar: Tek bir LLM client wrapper, runtime'da provider seçilebilir.

```
interface LLMProvider {
  name: 'claude' | 'openai' | 'groq';
  chat(messages: Message[], opts?: ChatOpts): Promise<Response>;
  structured<T>(prompt: string, schema: ZodSchema<T>): Promise<T>;
}

const llm = createLLM({ provider: env.LLM_DEFAULT, fallback: 'groq' });
```

Gerekçe: - **Vendor lock-in yok**: Anthropic pahalanırsa Groq'a geç - **Maliyet optimizasyonu**: kritik aksiyon Claude (kalite), bulk Groq (ucuz/hızlı) - **Fallback**: birinin API çökerse alternatife geç - Promats'a teslim: API key Promats'ın hesabında, kontrol onların

Provider seçim kuralları:

Görev	Önerilen	Neden
Konversasyonel sohbet	Claude	Reasoning kalitesi
Structured JSON çıkarımı	OpenAI gpt-4o-mini	Cheap + structured output
Bulk sinyal çıkarımı	Groq llama-3.1-70b	Hız + ucuz
Tahmin açıklama	Claude	Doğal dil kalite
OCR sonrası ekstrasyon	Claude	Karmaşık doc anlama

2.7 Mobil — Flutter

Karar: Saha satış + bayi mobil + operatör → Flutter.

Gerekçe: - Promats mobil projeleri zaten Flutter (Bereket Fide, VistaSeeds) - Tek codebase → iOS + Android - Performance native'a yakın - React Native: Bun ile çelişkiler, ayrı bilgi tabanı

Alternatif: - PWA (sadece bayi tarafı, Faz 9.7) — başlangıçta yeterli, sonradan native'a geçilebilir

2.8 Authentication — Fastify JWT + httpOnly cookie

Karar: Mevcut Paspas auth pattern'i tüm yeni modüllerde devam.

```
Login → POST /auth/login → JWT (15dk) + Refresh JWT (7gün) → httpOnly cookie
RTK Query her request'te cookie ekler
401 → otomatik refresh → tekrar dene
```

Gerekçe: - Mevcut Paspas auth çalışıyor, yeniden tasarlamaya gerek yok - httpOnly cookie: XSS koruma (localStorage'dan güvenli) - Refresh token rotation - Bayi portalı için: aynı auth mekanizması, farklı `audience` claim'i (bayi vs admin)

2.9 Cache & Queue — Redis + BullMQ

Karar: Tek Redis instance: cache + BullMQ queue + rate limit + session.

Kullanım: - **Cache:** tahmin sonucu (1 saat TTL), bayi cari (5dk TTL), katalog (15dk TTL) - **Queue:** scraper işleri, ML eğitim, email gönderim, Excel parsing - **Rate limit:** API per-IP, per-user - **Session:** opsiyonel — JWT zaten stateless

2.10 Object Storage — S3 / MinIO

Karar: Cloud deployment'ta AWS S3, self-host'ta MinIO.

Kullanım: - HTML snapshot'lar (Faz 5 — 90 gün retention) - Model artifact (Faz 7 — versiyon × bayi-ürün) - Bayi yüklenen dosyalar (iade fotoğraf, vs) - PDF rapor (cari ekstre, fatura)

3. Paspas ile Entegrasyon Noktaları

3.1 Mevcut Paspas tabloları (paylaşılan)

MatPortal aşağıdaki Paspas tablolarını **read** + bazılarına **write** yapar:

Tablo	Read	Write	Hangi modül?
`musteriler`	✅	⚠️ (sadece portal_kullanicilari linkler)	Tümü
`urunler`	✅	❌	Faz 6 (katalog)
`urun_kategorileri`	✅	❌	Faz 6
`satis_siparisleri`	✅	✅ (`repoCreate` çağırarak)	Faz 6
`siparis_kalemleri`	✅	✅	Faz 6
`uretim_emirleri`	✅	❌	Faz 6 (sipariş takip)
`sevkiyat_emirleri`	✅	❌	Faz 6
`hareketler`	✅	❌	Faz 6 (cari)
`stoklar`	✅	⚠️ (rezervasyon)	Faz 4, Faz 6
`recete_*`	✅	❌	Faz 4
`makine_havuzu`	✅	❌	Faz 2 (kapasite)
`vardiya_*`	✅	❌	Faz 2
`kullanicilar`	✅	❌	Tümü

Kritik prensip: MatPortal **doğrudan Paspas tablolarına yazmaz**. Yazma yapacaksa Paspas'ın repository fonksiyonlarını çağırır (örn. `satis_siparisleri/repository.ts:repoCreate`). Bu sayede: - Paspas iş kuralları korunur

(validation, audit) - Tek doğruluk kaynağı - İlerde Paspas API değişirse MatPortal kırılmaz, repo wrapper güncellenir

3.2 Yeni MatPortal tabloları

Faz bazlı yeni tablolar (özet):

```
Faz 1: talep_havuzu, talep_kalemleri, talep_kaynak
Faz 2: tahmin_modelleri, tahmin_calistirma, tahmin_dogruluk,
       siparis_gecmisi_yuklenen
Faz 3: lead_havuzu, lead_etkilesim, outreach_kampanya
Faz 5: web_kazima_hedefleri, web_kazima_kayitlari, bayi_sinyalleri,
       churn_skor_gecmisi, churn_aksiyon_log
Faz 6: portal_kullanicilari, portal_oturumlari, portal_sepet,
       portal_sepet_kalemleri, portal_audit, portal_bildirimler,
       portal_not_defteri, arac_modelleri, urun_arac_uyumlulugu,
       portal_numune_talepleri, portal_iadeler
Faz 7: model_versiyonlari, ab_test_calistirma, excel_yukleme_gecmisi,
       feature_pipeline_kayitlari, tahmin_geri_besleme
Faz 8: ai_aksiyon_audit, ai_otomasyon_ayarlari, ai_oneri_geri_besleme
Faz 9: ziyaret_kayitlari, push_bildirim_log, mobil_oturumlari
```

> **Detay:** `06-veri-modeli.md`

3.3 Mevcut sidebar genişlemesi

Paspas admin paneli sidebar’a yeni menü grupları:

```
PASPAS (mevcut)
  ├── Dashboard
  ├── Ürünler
  ├── Reçeteler
  ├── Müşteriler
  ├── Satış Siparişleri
  ├── Üretim Emirleri
  ├── Makine Havuzu
  ├── İş Yükleri
  ├── Gantt
  ├── Stoklar
  ├── Satın Alma
  ├── Hareketler
  ├── Operatör
  └── Tanımlar

MATPORTAL (yeni)
  ├── Talep Havuzu (Faz 1)
  ├── Tahmin Motoru (Faz 2)
  ├── Müşteri Keşif (Faz 3)
  ├── Stok Otomasyonu (Faz 4)
  ├── Bayi Radarı (Faz 5)
  ├── Bayi Portalı (Faz 6)
  │   ├── Bayi Yönetimi
  │   ├── Portal Raporları
  │   └── Araç-Paspas Uyumluluk
  ├── ML Laboratuvarı (Faz 7)
  └── Konversasyonel (Faz 8)
```

Yetki: Paspas admin, MatPortal admin, segment admin, satış sorumlusu rolleri. Her menü kendi yetkisine bağlı.

4. Deployment Topolojisi

4.1 Tek müşteri (Promats) deployment

```
                    ┌──────────────────┐
                    │  promats.local   │ → DNS
                    └──────────┬───────┘
                               ↓
                    ┌──────────────────┐
                    │  Nginx (TLS)     │
                    │  443 + 80→443    │
                    └──────────┬───────┘
                               ↓
        ┌──────────────────────┼──────────────────────┐
        ↓                      ↓                      ↓
┌────────────────┐    ┌────────────────────┐    ┌────────────────┐
│ Admin Panel    │    │ Bayi Portal        │    │ Backend API    │
│ Next.js        │    │ Next.js            │    │ Bun:3001       │
│ :3000          │    │ :3002              │    │                │
└────────┬───────┘    └──────────┬─────────┘    └────────┬───────┘
         │                       │                       │
         └───────────────────────┼───────────────────────┘
                                 ↓
                     ┌───────────┴──────────────┐
                     ↓                          ↓
        ┌────────────────┐          ┌────────────────┐
        │ MySQL :3306    │          │ Redis :6379    │
        │ promats_erp    │          │ cache+queue    │
        └────────────────┘          └────────────────┘
                     ↓
        ┌──────────────────────────┐
        │ ML Service (Docker)      │
        │ Python FastAPI :8000     │
        └──────────────────────────┘
                     ↓
        ┌──────────────────────────┐
        │ Crawler Worker (Docker)  │
        │ Bun + Playwright         │
        └──────────────────────────┘
                     ↓
        ┌──────────────────────────┐
        │ MinIO :9000              │
        │ (self-host) veya AWS S3  │
        └──────────────────────────┘
```

4.2 VPS gereksinimleri

Bileşen	RAM	CPU	Disk	Notlar
Ana VPS (backend + DB + admin/portal Next.js)	8GB	4 vCPU	100GB SSD	Faz 0-6 yeter
ML Service container	4GB	2 vCPU	50GB	Faz 7 sonrası
Crawler Worker	2GB	2 vCPU	20GB	Playwright RAM hungry
Redis	2GB	1 vCPU	10GB	Aynı VPS'te paylaşımlı olabilir
MinIO veya S3	10GB+	—	100GB+ büyür	HTML snapshot + model artifact

Önerilen başlangıç: 8GB+4vCPU+200GB tek VPS (~$30-50/ay). **Faz 7 itibariyle:** ML + Crawler ayrı container, ayrı host (~$80-120/ay).

4.3 Multi-deployment (gizli SaaS)

```
                    ┌──────────────────┐
                    │ Tek Codebase     │
                    │ (matportal-core) │
                    └──────────┬───────┘
                               ↓ deploy
          ┌────────────────────┼────────────────────┐
          ↓                    ↓                    ↓
┌────────────────┐   ┌────────────────┐   ┌────────────────┐
│ Promats        │   │ Müşteri 2      │   │ Müşteri N      │
│ matportal.     │   │ matportal-x.   │   │ matportal-y.   │
│ promats.com    │   │ com            │   │ com            │
│                │   │                │   │                │
│ DB: promats_   │   │ DB: musteri2_  │   │ DB: musteriN_  │
│     erp        │   │     erp        │   │     erp        │
│ Konfig: özel   │   │ Konfig: özel   │   │ Konfig: özel   │
│ LLM key: özel  │   │ LLM key: özel  │   │ LLM key: özel  │
└────────────────┘   └────────────────┘   └────────────────┘
```

Not: Bu strateji **Promats'a görünmez**. Promats için MatPortal "kendi özel ürünüdür". Ortak codebase Orhan'ın bilgisindedir, müşteri tarafına aksetmez.

Konfig farklılıkları: - `MATPORTAL_BRAND_NAME=Promats Bayi Portalı` - `MATPORTAL_SECTOR=plastik_enjeksiyon` - `MATPORTAL_SEASONALITY=yıllık` -

`MATPORTAL_CURRENCY=TRY` - `LLM_API_KEY=...` (her müşteri kendi)

5. Güvenlik

5.1 Authentication & Authorization

- JWT access token: **15 dk** TTL
- JWT refresh token: **7 gün** TTL, rotation aktif
- httpOnly cookie + SameSite=Strict
- Şifre: bcrypt (cost 12), min 10 karakter, complexity check
- 2FA (Faz 6 v2): TOTP veya SMS

5.2 Yetkilendirme matrisi

Rol	Paspas erişim	Portal erişim	ML Lab	Sohbet
Sistem admin	Tam	Tam	Tam	Tam
MatPortal admin	Bayi yönetim	Tam	Görüntüle	Tam
Satış sorumlusu	Müşteri+sipariş	Bayi yönetim	❌	Sınırlı
Bayi admin (portal-side)	❌	Kendi bayisi	❌	❌
Bayi operator	❌	Sipariş ver	❌	❌
Bayi görüntüleyici	❌	Read-only	❌	❌
Saha satış	Müşteri görüntüle	Atadığı bayiler	❌	Sınırlı

5.3 Veri koruma

- **At rest:** MySQL şifreli volume (LUKS), S3 SSE
- **In transit:** TLS 1.3 her yerde, HSTS aktif
- **PII:** bayi telefon/email yalnızca `musteriler` tablosunda — log'a girmez
- **KVKK:** aydınlatma metni, silme talebi flow'u (Faz 6)
- **Audit:** tüm sensitive aksiyonlar `*_audit` tablolarında, 7 yıl saklanır

5.4 Saldırı yüzeyi

Vektör	Önlem
SQL injection	Drizzle ORM parametreli sorgu (raw SQL yasak)
XSS	React 19 default escape + Content Security Policy
CSRF	SameSite=Strict cookie + CSRF token (form-based)
Brute force login	Rate limit 5 deneme/dakika + 30dk kilit
API rate limit	Redis bazlı per-IP + per-user limit
Secret leak	env değişkenleri Docker secret, repo'ya commit yok
LLM prompt injection	Kullanıcı girdisi `<user>` tag içinde, sistem prompt sabit
Scraping yasal sorun	robots.txt + KVKK + bayi sözleşme klozu (Faz 5)

6. Performans Hedefleri

6.1 Latency

Endpoint tipi	p50	p95	p99
Auth (login, refresh)	<100ms	<300ms	<500ms
CRUD basit	<80ms	<200ms	<400ms
Liste + filtre + sayfalama	<150ms	<500ms	<1s
Tahmin (cache hit)	<50ms	<100ms	<200ms
Tahmin (cache miss)	<500ms	<2s	<5s
LLM çağrısı	<2s	<8s	<15s
Excel upload (1000 satır)	<5s	<15s	<30s
Crawler bir sayfa	<3s	<10s	<20s

6.2 Throughput

Sistem	Hedef RPS	Notlar
Backend (peak)	200 RPS	Tek instance Bun + Fastify
Tahmin endpoint	50 RPS	Cache'li
Crawler worker	1 req / 2s / domain	Rate limit gereği

6.3 Ölçek hedefleri (1 yıl)

Metrik	Yıl 1	Yıl 2
Bayi sayısı	50-65	100+
Aktif kullanıcı (portal+admin)	~150	300+
Aylık sipariş	250-300	500+
Tahmin tablosu (kayıt)	~50K	200K
Bayi sinyalleri (kayıt)	~10K	50K
HTML snapshot (S3)	~5GB	30GB
Model artifact (S3)	~2GB	10GB

Bu rakamlar **tek VPS+tek Redis+tek MySQL** ile rahat karşılanır. 100+ bayide horizontal scaling değerlendirilebilir.

7. Geliştirme Akışı

7.1 Repository yapısı

```
paspas/                     # mevcut monorepo
├── admin_panel/            # mevcut Next.js admin
├── backend/                # mevcut Fastify backend (genişler)
│   ├── src/modules/
│   │   ├── musteriler/     # mevcut
│   │   ├── urunler/        # mevcut
│   │   ├── ...
│   │   ├── portal/         # Faz 6 yeni
│   │   ├── tahmin/         # Faz 2 yeni
│   │   ├── churn/          # Faz 5 yeni
│   │   ├── lead/           # Faz 3 yeni
│   │   ├── sohbet/         # Faz 8 yeni
│   │   ├── mlops/          # Faz 7 yeni
│   │   └── crawler/        # Faz 5 yeni
│   └── src/db/seed/sql/    # schema dosyaları
├── dealer_portal/          # Faz 6 yeni Next.js
├── ml_service/             # Faz 7 yeni Python container
├── crawler_worker/         # Faz 5 yeni Bun worker (ayrı süreç)
├── mobile/                 # Faz 9 yeni Flutter
└── docs/
    ├── tartisma/           # mevcut tartışma dokümanları
    └── proje-teklifi/      # mevcut teklif dokümanları
```

7.2 CI/CD

Pipeline: 1. PR açıldığında: lint + type-check + unit test 2. main'e merge: integration test + build 3. Auto-deploy staging 4. Manuel approve → production deploy

Stack: - GitHub Actions (Promats projeleri zaten kullanıyor) - Docker Compose (deployment) - Health check + auto-restart (Docker `restart: unless-stopped`)

7.3 Test piramidi

Tip	Coverage hedefi	Araç
Unit (kritik fonksiyonlar)	%80+	bun test
Integration (API + DB)	Kritik akışlar	bun test + test DB
E2E (kullanıcı senaryosu)	Top 10 senaryo	Playwright
Smoke (deploy sonrası)	Health check + login	Curl + GH Action

8. Operasyonel Konular

8.1 İzleme (monitoring)

Stack: - Application: Sentry (hata takibi) - Metrik: Prometheus + Grafana - Log: structured JSON → loki (ileride) - Uptime: UptimeRobot ya da Healthchecks.io (eksternal)

Kritik dashboard: - API latency p95/p99 - Hata oranı (HTTP 5xx) - DB connection pool - Crawler success rate - LLM token usage + maliyet - Tahmin MAPE (model dashboard) - Churn alarm count

8.2 Yedekleme

- MySQL: günlük tam dump → S3 (30 gün retention)
- MySQL: saatlik incremental binlog
- S3: cross-region replication (kritik artifact'lar)
- Konfig: Docker compose + env'ler git'te (secret hariç)

8.3 Disaster recovery

Senaryo	Recovery time hedefi	Yaklaşım
VPS çökmesi	<30 dk	Yeni VPS + dump restore
DB veri bozulması	<2 saat	Saatlik binlog restore
Model artifact kaybı	<4 saat	Re-train (offline)
Tüm S3 kaybı	<8 saat	Cross-region replica
Codebase kaybı	<15 dk	Git remote (GitHub/GitLab)

9. Açık karar noktaları (Mimari)

1. **Bun vs Node.js:** Bun stable mı yeterli, bazı paket uyumsuzluğu olabilir? (Önerim: Bun devam, sorun çıkan paketleri replace)
2. **MinIO vs AWS S3:** Self-host mu cloud mu? (Önerim: başlangıç MinIO, ölçek olunca S3)
3. **Mobil PWA vs native Flutter (bayi tarafı):** PWA ilk çıkışta yeter mi? (Önerim: PWA → ihtiyaç doğunca native)
4. **Microservice ayrımı:** Hangi noktada ML/Crawler'ı tamamen ayırmak gerek? (Önerim: 100+ bayi sonrası)
5. **MySQL vs PostgreSQL geçişi:** İlerde gerek olur mu? (Önerim: hayır, MySQL 8 yeterli)
6. **Kubernetes:** Ne zaman? (Önerim: 3+ müşteri deployment olunca)
7. **Frontend tek monorepo mu, ayrı repo mu (admin vs portal)?** (Önerim: monorepo, yarn workspaces / turborepo)
8. **WebSocket gerek mi (gerçek zamanlı bildirim)?** (Önerim: Faz 6'da Server-Sent Events; WS Faz 8'de)
9. **i18n (çoklu dil):** Faz 9 mu, baştan mı? (Önerim: anahtar yapısı baştan, çeviri Faz 9)
10. **GraphQL vs REST:** REST devam mı? (Önerim: REST devam, RTK Query iyi çalışıyor)